DC Lab — Experiment 7 Walkthrough (General Purpose)

Eventual Consistency via Gossip Replication + Last-Write-Wins

Goal: Implement a replicated data store where writes are accepted **locally and immediately** (favoring availability), then propagated to other replicas **asynchronously** in the background. When two replicas receive conflicting concurrent writes for the same key, resolve the conflict deterministically using **Last-Write-Wins**, based on a Lamport logical timestamp. Prove that all replicas **eventually converge** to the same value, even though they briefly disagreed.

Why this matters: Strong consistency (every replica agrees before any write is acknowledged) is safer but slower and less available — a write can be blocked if any replica is unreachable. Eventual consistency trades that safety for speed and availability: a client never waits on other replicas, and the system tolerates temporary disagreement as long as it's guaranteed to resolve once things settle down. This is the model behind real systems like DNS, many NoSQL databases (Cassandra, DynamoDB), and — relevant here — auto-save/sync features that must never block on the network.

Prerequisite: Concept-wise, this experiment builds directly on Lamport clocks (used here to order conflicting writes instead of RPC calls). An activated Python `venv` with `grpcio` + `grpcio-tools` installed.

Step 1 — Set up the project folder

```
mkdir dc_lab_exp7 && cd dc_lab_exp7
python3 -m venv venv
source venv/bin/activate
pip install grpcio grpcio-tools
mkdir proto
```

Step 2 — Define the service contract

`proto/replica.proto`:

```
syntax = "proto3";
package eventual_demo;

// Replicated key-value style store. Each replica exposes the same
// three RPCs to clients AND to its peer replicas.
service ReplicaService {
  rpc SaveValue (ValueUpdate) returns (SaveAck);   // client -> replica (local wr
  rpc SyncUpdate (ValueUpdate) returns (SaveAck);  // replica -> replica (gossip)
  rpc GetValue (ValueQuery) returns (ValueState);  // read current state
}

message ValueUpdate {
  string key = 1;
  string content = 2;
  int32 lamport_timestamp = 3;
  string origin_replica = 4;
}

message SaveAck {
  bool accepted = 1;
  string replica = 2;
  int32 lamport_timestamp = 3;
}

message ValueQuery {
  string key = 1;
}

message ValueState {
  string key = 1;
  string content = 2;
  int32 lamport_timestamp = 3;
  string origin_replica = 4;
}
```

Step 3 — Generate the Python stubs

```
python -m grpc_tools.protoc -I=proto --python_out=. --grpc_python_out=. proto/rep
```

Verify:

```
ls replica_pb2*.py
```

Step 4 — Understand the design before writing code

Three ideas combine here:

1. **Accept-then-gossip** — a replica's `SaveValue` handler applies the write to its own local store *first*, replies to the client *immediately*, and only *then* asynchronously notifies its peers. The client is never blocked waiting for replication.
2. **Lamport clock, same as always** — tick before sending anything, `max(local, received) + 1` on receiving anything. Here it's used to give every write a logical ordering, so replicas can later agree on which of two conflicting writes happened "later."
3. **Last-Write-Wins (LWW)** — when a replica receives an update (from a client or from a gossiping peer) for a key it already has a value for, it only overwrites if the incoming `(lamport_timestamp, origin_replica)` pair is strictly greater than what it currently stores. Otherwise, it discards the incoming update as stale. Comparing tuples `(ts, origin)` instead of just `ts` gives a deterministic tiebreak if two writes happen to get the same timestamp.

Step 5 — Write the replica (`replica_node.py`)

```
import sys, time, threading
from concurrent import futures
import grpc
import replica_pb2, replica_pb2_grpc

ALL_REPLICAS = {
    "A": "localhost:60301",
    "B": "localhost:60302",
    "C": "localhost:60303",
}


class ReplicaNode(replica_pb2_grpc.ReplicaServiceServicer):
    def __init__(self, name):
        self.name = name
        self.peers = {n: addr for n, addr in ALL_REPLICAS.items() if n != name}
        self.clock = 0
        self.lock = threading.Lock()
        self.store = {}   # key -> (content, lamport_timestamp, origin_replica)

    def log(self, msg):
        print(f"[{self.name} t={self.clock:>2}] {msg}", flush=True)
```

```
def tick(self):
    with self.lock:
        self.clock += 1
        return self.clock

def update_clock(self, received_ts):
    with self.lock:
        self.clock = max(self.clock, received_ts) + 1
        return self.clock

def _apply_if_newer(self, key, content, ts, origin):
    """Last-Write-Wins: only overwrite if the incoming update is
    strictly newer, tie-broken by origin replica name."""
    with self.lock:
        current = self.store.get(key)
        if current is None or (ts, origin) > (current[1], current[2]):
            self.store[key] = (content, ts, origin)
            self.log(f"APPLIED '{key}' = \"{content}\" (ts={ts}, origin={orig
            return True
        self.log(f"IGNORED stale update for '{key}' "
                 f"(incoming ts={ts}/{origin} <= current ts={current[1]}/{cur
        return False

def _gossip(self, key, content, ts, origin):
    """Fire-and-forget replication - runs AFTER the local write
    already succeeded and the client already got its response."""
    for peer_name, addr in self.peers.items():
        try:
            with grpc.insecure_channel(addr) as channel:
                stub = replica_pb2_grpc.ReplicaServiceStub(channel)
                stub.SyncUpdate(replica_pb2.ValueUpdate(
                    key=key, content=content,
                    lamport_timestamp=ts, origin_replica=origin))
                self.log(f"Gossiped '{key}' -> {peer_name}")
        except grpc.RpcError as e:
            self.log(f"Gossip to {peer_name} failed (will catch up later): {e

def SaveValue(self, request, context):
    ts = self.tick()
    self._apply_if_newer(request.key, request.content, ts, self.name)
    threading.Thread(
        target=self._gossip,
        args=(request.key, request.content, ts, self.name),
        daemon=True,
    ).start()
    return replica_pb2.SaveAck(accepted=True, replica=self.name, lamport_time
```

```
    def SyncUpdate(self, request, context):
        self.update_clock(request.lamport_timestamp)
        self._apply_if_newer(request.key, request.content,
                             request.lamport_timestamp, request.origin_replica)
        return replica_pb2.SaveAck(accepted=True, replica=self.name, lamport_time

    def GetValue(self, request, context):
        with self.lock:
            entry = self.store.get(request.key)
        if entry is None:
            return replica_pb2.ValueState(key=request.key, content="", lamport_ti
        content, ts, origin = entry
        return replica_pb2.ValueState(key=request.key, content=content,
                                      lamport_timestamp=ts, origin_replica=origi


def serve(name, port):
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=20))
    replica_pb2_grpc.add_ReplicaServiceServicer_to_server(ReplicaNode(name), serv
    server.add_insecure_port(f"localhost:{port}")
    server.start()
    print(f"Replica-{name} listening on localhost:{port}")
    try:
        while True:
            time.sleep(86400)
    except KeyboardInterrupt:
        server.stop(0)


if __name__ == "__main__":
    serve(sys.argv[1], int(sys.argv[2]))
```

Step 6 — Write the client (`replica_client.py`)

Deliberately creates a conflict — two “writers” hit two different replicas for the same key at nearly the same time — then checks convergence:

```
import time, threading, grpc
import replica_pb2, replica_pb2_grpc

REPLICAS = {"A": "localhost:60301", "B": "localhost:60302", "C": "localhost:60303


def save(replica_name, key, content):
    with grpc.insecure_channel(REPLICAS[replica_name]) as channel:
```

```
        stub = replica_pb2_grpc.ReplicaServiceStub(channel)
        ack = stub.SaveValue(replica_pb2.ValueUpdate(
            key=key, content=content, lamport_timestamp=0, origin_replica=""))
        print(f"[Client] Saved on Replica-{replica_name}: \"{content}\" "
              f"-> accepted={ack.accepted}, replica_ts={ack.lamport_timestamp}")


def read(replica_name, key):
    with grpc.insecure_channel(REPLICAS[replica_name]) as channel:
        stub = replica_pb2_grpc.ReplicaServiceStub(channel)
        state = stub.GetValue(replica_pb2.ValueQuery(key=key))
        return state.content, state.lamport_timestamp, state.origin_replica


def main():
    key = "shared-key-1"
    print("=== Simulating two concurrent writers hitting DIFFERENT replicas ===\n

    t1 = threading.Thread(target=save, args=("A", key, "value-from-writer-1"))
    t2 = threading.Thread(target=save, args=("C", key, "value-from-writer-2"))
    t1.start()
    time.sleep(0.05)   # near-simultaneous, not identical - creates a real race
    t2.start()
    t1.join()
    t2.join()

    print("\n=== Immediately after writes (replicas may still be mid-gossip) ==="
    for name in REPLICAS:
        content, ts, origin = read(name, key)
        print(f"Replica-{name}: \"{content}\" (ts={ts}, origin={origin})")

    print("\nWaiting 2s for gossip to finish propagating...\n")
    time.sleep(2)

    print("=== After convergence window ===")
    results = {}
    for name in REPLICAS:
        content, ts, origin = read(name, key)
        results[name] = content
        print(f"Replica-{name}: \"{content}\"")

    if len(set(results.values())) == 1:
        print(f"\n*** CONVERGED: all replicas agree on: \"{list(results.values())
    else:
        print(f"\n*** NOT YET CONVERGED: {set(results.values())} ***")
```

```
if __name__ == "__main__":
    main()
```

Step 7 — Run it (needs 4 terminals)

Terminals 1–3 — start the 3 replicas, one per terminal. Wait for each "listening" line before starting the next:

```
# Terminal 1
cd dc_lab_exp7 && source venv/bin/activate
python replica_node.py A 60301

# Terminal 2
cd dc_lab_exp7 && source venv/bin/activate
python replica_node.py B 60302

# Terminal 3
cd dc_lab_exp7 && source venv/bin/activate
python replica_node.py C 60303
```

Terminal 4 — once all three are listening, run the client:

```
cd dc_lab_exp7 && source venv/bin/activate
python replica_client.py
```

Step 8 — What to check in the output

- **The "immediately after" read should show disagreement** between replicas — some may have only seen one of the two writes yet. This isn't a bug; it's the actual inconsistency window eventual consistency accepts.
- **Each replica's own log** should show an `APPLIED` line for the write it received directly, then either another `APPLIED` (if a later-timestamped update arrives via gossip) or an `IGNORED stale update` (if an earlier-timestamped one arrives after a newer one is already stored).
- **The "after convergence window" read should show all replicas agreeing** — and specifically agreeing on the write with the **higher Lamport timestamp**, proving Last-Write-Wins picked the logically-later write, not just whichever arrived first physically.

Common mistakes

Mistake	Symptom	Fix
Gossiping before applying the local write	Client gets acknowledged before its own replica even has the value	Apply the write to local store first, THEN launch the gossip thread
Comparing only `lamport_timestamp` for LWW, not `(timestamp, origin)`	Two writes with the same timestamp cause replicas to disagree on which "won"	Always compare the full `(ts, origin)` tuple for a deterministic tiebreak
Checking convergence with no wait at all	Test always reports "not converged," even though the system is working correctly	Read once immediately (to show the real disagreement window), then again after a short wait
Making the client block until all replicas confirm	Defeats the entire point of eventual consistency (this becomes strong consistency)	`SaveValue` must return as soon as the *local* write succeeds, not after gossip completes
Writing to the *same* replica from both "writers"	No real conflict occurs, nothing to resolve	Deliberately target two *different* replicas for the same key to force contention